

TrioLink

Rede de sensores em cascata via MQTT sobre Yocto Linux (Raspberry Pi 5) — desafios do treinamento

Prazo de entrega: dezembro de 2026

Contexto

Este projeto é o treinamento prático de sistemas embarcados para o novo grupo. Ele é dividido em três desafios conectados: cada um de vocês fica responsável por um Raspberry Pi 5 rodando Linux embarcado (Yocto), e os três nós juntos formam uma cadeia de dados via MQTT. Além do nó próprio, cada aluno também é dono de uma peça de infraestrutura usada pelos outros dois — então além de sistemas embarcados, esse treinamento cobra alinhamento entre vocês três: se a peça de infraestrutura de alguém atrasar, os outros dois ficam travados.

Objetivo geral

Construir uma cascata de três nós que trocam dados via MQTT: cada nó publica sua leitura de sensor, o nó seguinte assina o dado do anterior, agrega com a própria leitura, e publica um dado mais "rico" adiante. O último nó consolida tudo e exibe o resultado final.

Requisitos obrigatórios (valem para os três nós)

- Imagem Yocto customizada para o Raspberry Pi 5, a partir da meta-raspberrypi.
- Integração do sensor via I2C — decisão justificada entre driver de kernel (subsistema IIO) e leitura em userspace (i2c-dev).
- Comunicação entre os nós via MQTT, com schema de payload combinado entre os três (formato, unidades, timestamp).
- Alguma forma de segurança no MQTT (TLS, usuário/senha ou PSK) — decisão do grupo, mas precisa existir e ser justificada.
- Cada nó deve continuar funcionando de forma sensata caso o dado de entrada (de outro nó) pare de chegar — não é aceitável travar.

Fluxo de dados (cascata MQTT)

Esta é a proposta inicial de tópicos — o formato exato do payload e a decisão de onde roda o broker Mosquitto ficam com vocês (ver seção "Fica por conta do grupo").

Tópico MQTT	Publicado por	Assinado por
sensor/bmp/raw	Nó 1 (BMP280)	Nó 2
sensor/mpu/raw	Nó 2 (MPU6050)	—
fusion/bmp_mpu	Nó 2	Nó 3
fusion/final	Nó 3	Interface / dashboard

Hardware por nó

Nó	Aluno	Sensor	SBC	Conectividade	Papel de infraestrutura
Nó 1	Aluno A	BMP280 (I2C)	Raspberry Pi 5 2GB	Wi-Fi / BT	BSP / Imagem Yocto compartilhada

Nó 2	Aluno B	MPU6050 (I2C)	Raspberry Pi 5 2GB	Wi-Fi / BT	Camada de comunicação MQTT
Nó 3	Aluno C	—	Raspberry Pi 5 2GB	Wi-Fi / BT	Consolidação / interface (TFT 2.4")

Desafio 1 — Nó BMP280 + BSP / Imagem Yocto

Responsável pelo nó com o sensor BMP280 (pressão, temperatura, altitude). Além de integrar o próprio sensor, este aluno é dono do meta-layer Yocto compartilhado: overlay de device tree para I2C, configuração de kernel, e a convenção de recipe usada para adicionar um sensor à imagem. Os outros dois nós constroem a própria imagem a partir desse layer — ele precisa estar em um repositório git acessível ao grupo desde cedo.

Desafio 2 — Nó MPU6050 + Camada de Comunicação

Responsável pelo nó com o sensor MPU6050 (acelerômetro e giroscópio). Além de integrar o próprio sensor e publicar/assinar os tópicos da cascata, este aluno é dono do schema MQTT e da segurança da comunicação, entregando uma biblioteca cliente simples (ex: uma função de publicar/assinar já com a segurança embutida) para os outros dois nós usarem em vez de cada um implementar o próprio wrapper MQTT.

Desafio 3 — Consolidação / Interface

Responsável pelo nó de consolidação, com o display TFT 2.4". Este aluno assina os dados agregados do Nó 2 (fusion/bmp_mpu), processa e gera o estado final do sistema, e é dono do framework de exibição/consolidação — a parte visível do sistema como um todo.

Fica por conta do grupo

As decisões abaixo são intencionalmente parte do desafio e devem ser combinadas entre os três antes de cada um sair implementando por conta própria.

- Onde roda o broker Mosquitto (e o que acontece se esse nó cair).
- Formato exato do payload JSON (campos, unidades, timestamp).
- Driver de kernel vs. userspace para cada sensor.
- Regra de fusão dos dados no Nó 2 e no Nó 3 (o que conta como estado "anormal", se for o caso).
- Mecanismo de segurança do MQTT (TLS, usuário/senha, PSK).
- Como cada nó se comporta quando o dado de outro nó para de chegar.

Entregáveis

- Código-fonte e camada Yocto de cada nó, com o meta-layer compartilhado versionado em git.
- Biblioteca/wrapper MQTT compartilhado, com schema documentado.
- Demonstração dos três nós funcionando em cascata ao vivo.
- Documento curto (pode ser informal) explicando as decisões de arquitetura tomadas em grupo.